

Voice Booking Agent Blueprint

Overview

This document is a developer-ready blueprint for a multilingual phone booking system that works for both:
- mobile car detailing businesses - cleaning businesses

It is designed to support: - English - Hindi - Mandarin - keypad options and speech input - quotes - availability checks - booking creation - cancellations - reminders - provider summary emails at 4:30 PM Melbourne time

Recommended Architecture

Core stack

- Twilio Voice for inbound calls, DTMF keypad input, speech input, and TwiML call control
- n8n for webhook handling, workflow orchestration, booking logic, reminders, and summaries
- PostgreSQL as the source-of-truth database
- Google Calendar for availability checking and booking event creation
- Optional AI layer only for multilingual understanding, transcript cleanup, and off-script fallback

Design principle

Use AI only where it adds value: - language understanding - transcript cleanup - natural phrasing in different languages - fallback handling for unclear speech

Do not use AI for: - quote calculation - slot generation - booking creation - booking cancellation - reminder timing - business rules

Frontend and Backend

Frontend

Two interfaces are recommended.

1. Customer-facing web pages

- instant quote page
- booking confirmation page
- cancel booking page
- reschedule booking page
- optional chat widget
- optional click-to-call button

2. Provider dashboard

- dashboard
- bookings
- tomorrow's bookings
- calls
- callbacks
- services
- pricing
- availability
- settings

Backend

The backend is divided into: - telephony/API layer - booking orchestration layer - notification layer

Telephony/API layer

- receives Twilio webhook requests
- reads current call state
- returns TwiML XML

Booking orchestration layer

- loads business config
- loads service config
- validates inputs
- calculates quote
- checks availability
- creates booking
- cancels or reschedules booking

Notification layer

- customer confirmation
- provider confirmation
- customer reminder
- provider day-before reminder
- provider summary email at 4:30 PM

PostgreSQL Schema

businesses

```
CREATE TABLE businesses (  
  business_id UUID PRIMARY KEY,  
  business_name TEXT NOT NULL,  
  business_type TEXT NOT NULL,
```

```
    timezone TEXT NOT NULL DEFAULT 'Australia/Melbourne',
    provider_email TEXT NOT NULL,
    twilio_number TEXT,
    calendar_id TEXT,
    active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

services

```
CREATE TABLE services (
    service_id UUID PRIMARY KEY,
    business_id UUID NOT NULL,
    keypad_option INT NOT NULL,
    service_code TEXT NOT NULL,
    name_en TEXT NOT NULL,
    name_hi TEXT,
    name_zh TEXT,
    duration_minutes INT NOT NULL,
    base_price_min NUMERIC(10,2),
    base_price_max NUMERIC(10,2),
    manual_review_required BOOLEAN NOT NULL DEFAULT FALSE
);
```

customers

```
CREATE TABLE customers (
    customer_id UUID PRIMARY KEY,
    full_name TEXT,
    phone TEXT NOT NULL,
    email TEXT,
    preferred_language TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

bookings

```
CREATE TABLE bookings (
    booking_id UUID PRIMARY KEY,
    business_id UUID NOT NULL,
    customer_id UUID NOT NULL,
    service_id UUID NOT NULL,
    status TEXT NOT NULL,
    scheduled_start TIMESTAMP,
```

```

scheduled_end TIMESTAMP,
address_line TEXT,
suburb TEXT,
quote_min NUMERIC(10,2),
quote_max NUMERIC(10,2),
final_price NUMERIC(10,2),
notes TEXT,
calendar_event_id TEXT,
created_at TIMESTAMP NOT NULL DEFAULT NOW(),
updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

calls

```

CREATE TABLE calls (
  call_id UUID PRIMARY KEY,
  business_id UUID,
  twilio_call_sid TEXT UNIQUE,
  caller_phone TEXT,
  language_code TEXT,
  current_state TEXT,
  transcript TEXT,
  intent TEXT,
  selected_service_code TEXT,
  booking_id UUID,
  outcome TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

booking_events

```

CREATE TABLE booking_events (
  event_id UUID PRIMARY KEY,
  booking_id UUID NOT NULL,
  event_type TEXT NOT NULL,
  event_payload JSONB,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

availability_rules

```

CREATE TABLE availability_rules (
  rule_id UUID PRIMARY KEY,

```

```

business_id UUID NOT NULL,
weekday INT NOT NULL,
start_time TIME NOT NULL,
end_time TIME NOT NULL,
slot_minutes INT NOT NULL DEFAULT 30,
active BOOLEAN NOT NULL DEFAULT TRUE
);

```

notification_log

```

CREATE TABLE notification_log (
  notification_id UUID PRIMARY KEY,
  booking_id UUID,
  recipient_type TEXT NOT NULL,
  channel TEXT NOT NULL,
  subject TEXT,
  body TEXT,
  sent_at TIMESTAMP,
  status TEXT
);

```

callback_requests

```

CREATE TABLE callback_requests (
  callback_id UUID PRIMARY KEY,
  business_id UUID,
  phone TEXT,
  language_code TEXT,
  reason TEXT,
  status TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

Universal Business Config

Use one config per business.

Example config

```

{
  "business_type": "car_detailing",
  "languages": ["en", "hi", "zh"],
  "provider_daily_summary_time": "16:30",

```

```

"customer_reminder_hours_before": 24,
"provider_reminder_hours_before": 24,
"main_menu": {
  "1": "new_booking",
  "2": "check_availability",
  "3": "get_quote",
  "4": "change_booking",
  "5": "cancel_booking",
  "6": "leave_message"
}
}

```

Quote Engine

Car detailing inputs

- service_type
- vehicle_type
- add_ons
- suburb
- vehicle_condition
- mobile_or_dropoff

Cleaning inputs

- service_type
- property_type
- bedrooms
- bathrooms
- extras
- suburb
- frequency

Quote engine rules

- read base price range from service rules
- apply add-on pricing
- apply suburb or travel fee
- apply condition or frequency multiplier if needed
- return quote_min and quote_max
- flag manual review if required

Availability Engine

Inputs: - business_id - service_id - requested date window - duration_minutes - provider calendar_id - availability rules

Logic: - read provider working hours - read service duration - query Google Calendar free/busy - remove busy times from available blocks - generate top 3 valid slots

Output: - slot_start - slot_end - spoken labels in English, Hindi, and Mandarin if needed

Booking Engine

Steps: - validate required fields - recheck slot is still available - create booking in PostgreSQL - create Google Calendar event - save calendar_event_id - log booking event - notify provider - notify customer

Cancellation and Reschedule Engine

Cancellation

- identify booking by phone and future booking
- confirm cancellation
- set booking status to cancelled
- update or delete Google Calendar event
- notify provider
- notify customer
- offer rebooking

Reschedule

- identify booking
- generate next available slots
- confirm selected slot
- update booking row
- update Google Calendar event
- notify provider
- notify customer

Twilio Route Table

Main states

- LANGUAGE_MENU
- MAIN_MENU_EN
- MAIN_MENU_HI
- MAIN_MENU_ZH
- SERVICE_MENU_NEW
- COLLECT_REQUIRED_FIELDS
- QUOTE_RESULT
- CHECK_AVAILABILITY
- CONFIRM_SLOT
- CREATE_BOOKING
- IDENTIFY_BOOKING_CANCEL

- CANCEL_CONFIRM
- CANCEL_BOOKING
- CALLBACK_REQUEST
- END_CALL

LANGUAGE_MENU

Options: - 1 English - 2 Hindi - 3 Mandarin - 9 Repeat

Transitions: - 1 -> MAIN_MENU_EN - 2 -> MAIN_MENU_HI - 3 -> MAIN_MENU_ZH - 9 -> LANGUAGE_MENU

MAIN_MENU_EN

Options: - 1 new booking - 2 check availability - 3 get quote - 4 change booking - 5 cancel booking - 6 leave message / callback - 9 repeat

Transitions: - 1 -> SERVICE_MENU_NEW - 2 -> SERVICE_MENU_NEW or availability flow - 3 -> SERVICE_MENU_NEW or quote flow - 4 -> identify booking - 5 -> identify booking cancel - 6 -> callback request - 9 -> MAIN_MENU_EN

SERVICE_MENU_NEW

Dynamic from services table.

Car detailing example: - 1 exterior wash - 2 interior detail - 3 full detail - 4 pre-sale detail - 5 ceramic coating enquiry - 6 other service

Cleaning example: - 1 regular cleaning - 2 deep cleaning - 3 end-of-lease cleaning - 4 Airbnb turnover - 5 office cleaning - 6 other service

COLLECT_REQUIRED_FIELDS

Ask one missing field at a time.

Car detailing required fields: - vehicle type - make/model - suburb - mobile/drop-off - condition - add-ons - preferred date - preferred time window - name - email

Cleaning required fields: - property type - bedrooms - bathrooms - suburb - extras - preferred date - preferred time window - name - email

QUOTE_RESULT

Read quote range and offer: - 1 check availability - 2 send quote only - 3 return to main menu - 9 repeat

CHECK_AVAILABILITY

Offer three slots: - 1 slot 1 - 2 slot 2 - 3 slot 3 - 4 hear more - 9 repeat

CONFIRM_SLOT

- 1 confirm booking
- 2 hear other times
- 3 cancel request

CREATE_BOOKING

- create booking row
- create Google Calendar event
- send confirmations
- end call with success message

IDENTIFY_BOOKING_CANCEL

- lookup future booking by caller phone
- read back booking details
- 1 continue
- 2 main menu

CANCEL_CONFIRM

- 1 cancel booking
- 2 keep booking
- 3 hear details again

CANCEL_BOOKING

- update DB status
- update or delete calendar event
- notify both sides
- offer new booking

CALLBACK_REQUEST

- collect spoken callback reason
- save callback request row
- notify provider

Twiml Templates

LANGUAGE_MENU

```
<Response>
  <Gather input="dtmf speech" numDigits="1" timeout="5" action="https://
  YOURDOMAIN/webhook/twilio-router" method="POST">
    <Say>Welcome. For English, press 1. Hindi ke liye 2 dabaiye. 中文请按 3. To
    repeat this menu, press 9.</Say>
```

```
</Gather>
<Redirect method="POST">https://YOURDOMAIN/webhook/twilio-router</Redirect>
</Response>
```

MAIN_MENU_EN

```
<Response>
  <Gather input="dtmf speech" numDigits="1" timeout="5" action="https://
YOURDOMAIN/webhook/twilio-router" method="POST">
    <Say>To make a new booking, press 1. To check availability, press 2. To get
a quote, press 3. To change a booking, press 4. To cancel a booking, press 5.
To leave a message or request a callback, press 6. To repeat, press 9.</Say>
  </Gather>
  <Redirect method="POST">https://YOURDOMAIN/webhook/twilio-router</Redirect>
</Response>
```

MAIN_MENU_HI

```
<Response>
  <Gather input="dtmf speech" numDigits="1" timeout="5" action="https://
YOURDOMAIN/webhook/twilio-router" method="POST">
    <Say language="hi-IN">नई बुकिंग के लिए 1 दबाइए। उपलब्ध समय जानने के लिए 2 दबाइए। कोटेशन
के लिए 3 दबाइए। बुकिंग बदलने के लिए 4 दबाइए। बुकिंग रद्द करने के लिए 5 दबाइए। संदेश छोड़ने या कॉल बैक
के लिए 6 दबाइए। दोबारा सुनने के लिए 9 दबाइए।</Say>
  </Gather>
  <Redirect method="POST">https://YOURDOMAIN/webhook/twilio-router</Redirect>
</Response>
```

MAIN_MENU_ZH

```
<Response>
  <Gather input="dtmf speech" numDigits="1" timeout="5" action="https://
YOURDOMAIN/webhook/twilio-router" method="POST">
    <Say language="cmn-CN">新预约请按1，查询可预约时间请按2，获取报价请按3，更改预约请按
4，取消预约请按5，留言或请求回电请按6。重听请按9。</Say>
  </Gather>
  <Redirect method="POST">https://YOURDOMAIN/webhook/twilio-router</Redirect>
</Response>
```

CHECK_AVAILABILITY

```
<Response>
  <Gather input="dtmf speech" numDigits="1" timeout="6" action="https://
```

```
YOURDOMAIN/webhook/twilio-router" method="POST">
  <Say>I found three available times. For tomorrow at 10 A M, press 1. For
tomorrow at 1 P M, press 2. For tomorrow at 4 P M, press 3. To hear more times,
press 4. To repeat, press 9.</Say>
  </Gather>
  <Redirect method="POST">https://YOURDOMAIN/webhook/twilio-router</Redirect>
</Response>
```

CANCEL_CONFIRM

```
<Response>
  <Gather input="dtmf speech" numDigits="1" timeout="5" action="https://
YOURDOMAIN/webhook/twilio-router" method="POST">
    <Say>To cancel your booking, press 1. To keep your booking, press 2. To
hear the booking details again, press 3.</Say>
  </Gather>
  <Redirect method="POST">https://YOURDOMAIN/webhook/twilio-router</Redirect>
</Response>
```

n8n Workflows

Workflow 1: twilio-incoming-call-router

Nodes: 1. Webhook 2. Set - Normalize Twilio Payload 3. Postgres - Lookup Call By CallSid 4. IF - Call Exists? 5. Postgres - Create Call Row 6. Set - Load Active Call Context 7. Switch - Route By Current State 8. Code - Build LANGUAGE_MENU TwiML 9. Code - Build MAIN_MENU_EN TwiML 10. Code - Build MAIN_MENU_HI TwiML 11. Code - Build MAIN_MENU_ZH TwiML 12. Postgres - Load Service Menu 13. Code - Build SERVICE_MENU TwiML 14. Code - Parse User Selection 15. Postgres - Update Call State 16. Execute Workflow - Quote And Availability 17. Code - Build QUOTE_RESULT TwiML 18. Code - Build CHECK_AVAILABILITY TwiML 19. Execute Workflow - Create Booking 20. Code - Build BOOKING_CONFIRMED TwiML 21. Execute Workflow - Cancel Booking 22. Code - Build CANCELLED_SUCCESS TwiML 23. Code - Build CALLBACK_REQUEST TwiML 24. Respond to Webhook

Workflow 2: quote-and-availability

- load business config
- validate fields
- calculate quote
- check Google Calendar free/busy
- generate 3 slots
- return quote + slots

Workflow 3: create-booking

- recheck slot availability
- insert booking row

- create Google Calendar event
- save calendar_event_id
- send provider confirmation
- send customer confirmation

Workflow 4: cancel-booking

- verify booking
- set status cancelled
- update calendar event
- notify provider
- notify customer

Workflow 5: customer-reminders

- schedule trigger hourly
- find bookings starting in ~24 hours
- send customer reminder
- log notification

Workflow 6: provider-summary-1630

- schedule trigger daily at 4:30 PM Melbourne time
- fetch tomorrow bookings
- count total
- sum estimates if needed
- email provider summary

n8n Router Build Sheet

Webhook node settings

- method: POST
- path: twilio-router
- response mode: using Respond to Webhook node

Normalize Twilio Payload fields

- call_sid
- from_phone
- to_phone
- digits
- speech_result
- confidence
- call_status

Selection parser code

```
const digits = ($json.digits || "").trim();
const speech = ($json.speech_result || "").trim().toLowerCase();
const state = $json.current_state;

let selected = null;

if (digits) selected = digits;

if (!selected && state === "LANGUAGE_MENU") {
  if (speech.includes("english")) selected = "1";
  else if (speech.includes("hindi")) selected = "2";
  else if (speech.includes("mandarin") || speech.includes("chinese")) selected
= "3";
}

return [{
  json: {
    ...$json,
    selected_option: selected || ""
  }
}];
```

Twiml builder template

```
const sayText = $json.say_text || "Sorry, please try again.";
const actionUrl = "https://YOURDOMAIN/webhook/twilio-router";

const xml = `<?xml version="1.0" encoding="UTF-8"?>
<Response>
  <Gather input="dtmf speech" numDigits="1" timeout="5" action="${actionUrl}"
method="POST">
    <Say>${sayText}</Say>
  </Gather>
  <Redirect method="POST">${actionUrl}</Redirect>
</Response>`;

return [{ json: { xml } }];
```

Provider Summary Logic

At 4:30 PM Melbourne time every day: - fetch all bookings for tomorrow - count total number of bookings - group by service type - sum quote ranges if desired - email service provider summary

Recommended MVP Scope

Launch first with: - English only - new booking - quote - availability - booking creation - cancellation - provider confirmation email - customer confirmation - 24-hour reminders - 4:30 PM tomorrow summary

Then add: - Hindi - Mandarin - callback capture - reschedule flow - AI fallback for messy speech

Implementation Order

1. PostgreSQL schema
2. business config
3. service catalog
4. quote engine
5. availability engine
6. booking engine
7. cancellation engine
8. Twilio router
9. provider notifications
10. customer reminders
11. provider summary automation
12. dashboard and customer pages
13. AI fallback layer

Final Recommendation

The most reliable production path is: - deterministic booking logic first - Twilio keypad + speech call tree second - multilingual expansion third - AI fallback last

This keeps the system accurate, debuggable, and scalable for both car detailing and cleaning businesses.